Invoice Information Pipeline: design and decision note

Problem
Extract fields from invoices into a structured table, validate totals and dates, send uncertain or wrong extractions to human review, and report accuracy. Synthetic data, Python and SQL, no paid API, no secrets.

Approach
Generate 100 seeded synthetic invoices (text and text-based PDF) with ground truth, then run ingest, extract, validate, route and store in SQLite. Evaluation compares the run with ground truth and writes Markdown and JSON reports.

Decisions that matter
- Swappable extractor: one interface; regex baseline and an LLM extractor behind a one-method client. The pipeline never imports a concrete class.
- Extraction and validation are separate: the extractor reads, validators judge. Validators are the second opinion because confidence can be wrong.
- One rules module: severity, tolerance, duplicate logic and the review decision are shared by the generator and the validator, so ground truth cannot silently disagree with detection.
- Wrong approvals cost more than extra reviews: any failed check routes to review. Skipped checks never approve; MISSING_FIELD is never skipped.
- Read failure is not garbled text: an unreadable file is listed as failed (not queued, never approved); readable but junk content is flagged by validation.
- Money is Decimal in Python and integer paise in SQLite; fractional paise are rejected.
- Ground truth lives in separate tables; expected and detected defects are never mixed.
- Each document is saved in one transaction; a failing document is recorded and the batch continues.

Results (synthetic data)
Metrics are kept separate, never one accuracy number. Development layouts A-D: regex exact match 94/94, 0 bad auto-approvals of 51, review recall 49/49, false review rate 0/51. These are circular: the same author wrote the generator and the regex. Held-out layout E (unseen labels and order): exact match 0/94, 0 wrong values, but 7 fields left empty, so all 100 documents went to review and none was auto-approved. Safe, but not useful. Confidently wrong scripted extractors: a wrong total is always caught by arithmetic; a wrong vendor is not caught (51 of 94 auto-approved) because no check examines the vendor.

Trade-offs
Review rate versus safety: the validators favour review. On an unseen layout that gives 100% review, which is the correct failure mode but offers no automation. Fixed thresholds (garbled text, confidence 0.80) were set before looking at results, so GARBLED_TEXT does not fire for character-swap or truncated documents; other checks catch them. This is reported as a finding, not tuned away.

Limitations
Data is synthetic and real invoices will do worse. No real LLM was evaluated: the only client is a scripted mock that answers from ground truth and verifies plumbing, not accuracy. Each defect type has 3-5 examples. Cancelling errors (consistent wrong subtotal and total) and unchecked fields (vendor) can pass. No OCR; Indian digit grouping is unsupported.

Next steps
Local LLM client (for example Ollama) behind the existing interface; vendor check against a known-vendor list; OCR for scanned PDFs; learning from reviewer corrections; fuzzy vendor matching for duplicates.